MODULE 16 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Functions I: def, return and None

Defining and calling functions, return versus print, the None every function can give back, and two useful modules.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 16 of 28

admin@sthithapragna.com

A function gives back exactly what its **return** statement hands it. With no return, it gives back **None**. Printing is not returning.

def creates the function. Calling it runs the body

```
1  def greet(name):              # header: def, name, parameters, colon
2      return "Hello, " + name    # body: indented
3
4  message = greet("Ann")        # the call runs the body
5  print(message)                # Hello, Ann
```

1 def, a name, brackets holding the parameters, and a colon. The brackets are required even when empty.

4 Calling the function runs its body, and the call is replaced by the returned value.

def fun: is a SyntaxError, expected '('. A parameterless function is written def fun():

The def must run before the call

```
1  def a():
2      return b() + 1          # b is only looked up when a runs
3
4  def b():
5      return 5
6
7  print(a())                  # 6: b exists by the time a is called
```

2 A name inside a body is looked up when the body runs, not when the def runs.

7 By this line, both defs have run, so both functions exist.

Calling a function above its def is a NameError: print(hello()) followed by def hello() fails.

return hands a value back. print only shows it

PRINT

Shows text; gives back None

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

6, then None. show prints 6, but returns nothing, so y is None.

RETURN

Gives back the value

```
1  def calc(x):
2      return x * 2
3
4  y = calc(3)
5  print(y)
```

6, once. calc prints nothing; the value comes back and is stored in y.

THE TRAP

print(func()) prints whatever func prints, then func's return value. For a printing function, that ends in None.

Three ways to return None

```
1  def nothing():
2      pass                    # no return at all
3  def bare():
4      return                  # return with no value
5  def maybe(n):
6      if n > 10:
7          return "big"        # only some paths return
8  print(nothing(), bare(), maybe(5))    # None None None
```

2 Falling off the end of the body returns None.

4 A bare return ends the function and returns None.

5-7 If a path through the function never reaches a return, that call gives None.

Nothing after a return runs

```
1  def f():
2      return 1
3      print("never")          # unreachable
4  def first_even(nums):
5      for n in nums:
6          if n % 2 == 0:
7              return n        # leaves the loop and the function
8      return -1
9  print(f(), first_even([3, 8, 6]), first_even([1, 5]))   # 1 8 -1
```

3 Unreachable code is legal, but it never runs.

7 return inside a loop ends the loop and the function together.

An expression on its own line changes nothing

```
1  def grow(n):
2      n * 10                    # worked out, then thrown away
3      return n + 1
4
5  print(grow(grow(2)))          # grow(2) is 3, grow(3) is 4
```

2 Nothing assigns the result, so n is unchanged. It would need n = n * 10.

5 Nested calls run from the inside out.

Functions calling functions, and returning several values

```
1  def double(x): return x * 2
2  def add3(x): return x + 3
3  print(double(add3(1)), add3(double(1)))   # 8 5
4  def pair():
5      return 1, 2              # packed into one tuple
6  print(pair(), type(pair()))  # (1, 2) <class 'tuple'>
```

1-2 A one-line body can follow the colon directly, just as with if.

3 Inside out: add3(1) is 4, doubled is 8. double(1) is 2, plus 3 is 5.

5 return 1, 2 returns one tuple. The caller can unpack it: a, b = pair().

What goes wrong with functions

CODE	ERROR
`print(hello()) above def hello():`	NameError: name 'hello' is not defined
`print(func4(1)) when only func1 exists`	NameError. Did you mean: 'func1'?
`def fun:`	SyntaxError: expected '('
`def 1function():`	SyntaxError: a name cannot start with a digit
`def f(a, b): ... f(1)`	TypeError: missing 1 required positional argument: 'b'
`def f(): ... f(5)`	TypeError: f() takes 0 positional arguments but 1 was given

THE TIEBREAKER

A bad definition fails before anything runs. A bad call fails only when that line is reached.

Shadowing a built-in function

SHADOWED

list now names your list

```
1  list = [1, 2]
2  print(list("ab"))
```

TypeError: 'list' object is not callable. The built-in is hidden.

NOT SHADOWED

list is still the built-in

```
1  items = [1, 2]
2  print(list("ab"))
```

['a', 'b']. Choose a name that does not hide a built-in.

ON THE EXAM

Built-in functions such as print, len, list and input are names, not keywords. Assigning to them hides them.

import math

```
1  import math
2  print(math.sqrt(16), math.pi)          # 4.0 3.141592653589793
3  print(math.floor(-2.5), math.ceil(2.1))   # -3 3
4  print(math.pow(2, 3))                  # 8.0: always a float
5  from math import sqrt
6  print(sqrt(25))                        # 5.0: no math. prefix needed
```

1-4 import math brings in the module. Its contents are used as math.name.

3 floor rounds down, ceil rounds up. Both return int.

5-6 from math import sqrt copies just that name in, so it is used bare.

sqrt(4) without from math import sqrt is a NameError. math.sqrt(-1) is a ValueError.

import random

```
1  import random
2  n = random.randint(1, 6)       # 1 to 6, both ends included
3  x = random.random()            # a float from 0.0 up to, not including, 1.0
4  c = random.choice(["a", "b"])  # one element of the sequence
5  import math as m
6  print(m.sqrt(9))               # 3.0: an alias for the module
```

2 randint includes both ends, unlike range. randint(1, 6) can return 6.

5 import ... as gives the module a shorter name.

Python 3.14 reworded one maths error

- Python 3.12: math.sqrt(-1) says **ValueError: math domain error**.
- Python 3.14: **ValueError: expected a nonnegative input, got -1.0**.
- The exception is ValueError in both. Answer by the exception name.

What to carry into the exam

- A call is replaced by the **returned value**. No return, or a bare return: **None**.
- **print is not return.** print(func()) shows func's printing, then its return value.
- return ends the function at once, loops included.
- The def must run before the call; names in a body are looked up when it runs.
- n * 10 on its own line changes nothing. randint includes both ends.

What is the output of this program?

```
1  def check(n):
2      if n > 10:
3          return "big"
4
5  result = check(5)
6  print(result, check(20))
```

A `None big`

B `big big`

C The code raises a TypeError

D `small big`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def check(n):
2      if n > 10:
3          return "big"
4
5  result = check(5)
6  print(result, check(20))
```

check(5) never reaches a return, so it gives None.

A `None big`

5 skips the only return, so None; 20 returns "big".

B `big big`

5 > 10 is False, so the return is never reached for 5.

C The code raises a TypeError

Missing a return path is legal. It just gives None.

D `small big`

There is no "small" anywhere. Nothing makes one up.

THE SIGNAL

If some path through a function has no return, that call gives None.

What is the output of this program?

```
1  def first_even(nums):
2      for n in nums:
3          if n % 2 == 0:
4              return n
5      return -1
6
7  print(first_even([3, 8, 6]), first_even([1, 5]))
```

A. 6 -1

B. 8 None

C. 8 -1

D. [8, 6] -1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def first_even(nums):
2      for n in nums:
3          if n % 2 == 0:
4              return n
5      return -1
6
7  print(first_even([3, 8, 6]), first_even([1, 5]))
```

8 is the first even number, and return ends the loop there. [1, 5] has none.

A `6 -1`

Keeps looping after 8. return ends the function at once.

B `8 None`

Misses the final return -1 after the loop.

C `8 -1`

First even is returned; no even at all reaches return -1.

D `[8, 6] -1`

return hands back one value and stops. It does not collect.

THE SIGNAL

return inside a loop ends the loop and the function together, at the first match.

What is the output of this program?

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

A 6, then 6

B None, then 6

C 6 only

D 6, then None

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

Line 4 runs show, which prints 6. show returns None, so y is None.

A 6, then 6

Treats print as return. show gives back nothing.

B None, then 6

Reverses the order. The call on line 4 prints first.

C 6 only

Forgets that print(y) runs, and shows None.

D 6, then None

show prints 6 during the call, then returns None into y.

THE SIGNAL

Write down what a function prints, then what it returns, separately.

What is the output of this program?

```
1  def grow(n):
2      n * 10
3      return n + 1
4
5  print(grow(grow(2)))
```

A 211

B 31

C The code raises a SyntaxError

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grow(n):
2      n * 10
3      return n + 1
4
5  print(grow(grow(2)))
```

n * 10 is thrown away. grow(2) is 3, then grow(3) is 4.

A 211

*Treats n * 10 as updating n twice. Nothing is assigned.*

B 31

*Applies the * 10 once. It never applies.*

C The code raises a SyntaxError

An expression on its own line is legal; it is simply discarded.

D 4

Only return n + 1 has an effect: 2 to 3, then 3 to 4.

THE SIGNAL

A calculation that is not assigned, returned or printed has no effect.

What is the output of this program?

```
1  def a():
2      return b() + 1
3
4  def b():
5      return 5
6
7  print(a())
```

A The code raises a NameError

B 6

C 5

D None

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def a():
2      return b() + 1
3
4  def b():
5      return 5
6
7  print(a())
```

Both defs have run by line 7. b is looked up only when a's body runs.

A The code raises a NameError

b is defined before a is called. The body's names are looked up then.

B 6

a calls b, which returns 5, and adds 1.

C 5

Forgets the + 1 in a.

D None

Both functions return values.

THE SIGNAL

A def must run before a call to it runs. The order of defs among themselves does not matter.

> Consider Python functions in general.

Which two statements about functions are true? (Choose two.)

A. A function with no return statement returns None

B. return must be the last line of a function body

C. A function must be defined before it is called

D. print() inside a function sends a value back to the caller

◆ PAUSE AND DECIDE

Which two statements about functions are true? (Choose two.)

A. A function with no return statement returns None

Falling off the end gives None.

B. return must be the last line of a function body

return can appear anywhere, several times; later lines are just unreachable.

C. A function must be defined before it is called

The def must have run before the call runs, or it is a NameError.

D. print() inside a function sends a value back to the caller

print only shows text. return sends a value back.

THE SIGNAL

None when nothing is returned; def before the call; print shows, return gives back.